

We use some essential cookies to make our website work.

We use optional cookies, as detailed in our [cookie policy](#), to remember your settings and understand how you use our website.

[Accept optional cookies](#)

[Reject optional cookies](#)

Documentation

or [Ask](#)

Computers

[Getting started](#)

[Raspberry Pi OS](#)

[Configuration](#)

[config.txt](#)

[The Linux kernel](#)

Remote access

[Introduction to remote access](#)

[Find the IP address of your Raspberry Pi](#)

[Access a remote terminal with SSH](#)

[Screen share with VNC](#)

[Remote access with Raspberry Pi Connect](#)

[Share files with SCP](#)

[Synchronise folders between computers with rsync](#)

[Network File System \(NFS\)](#)

[Samba \(SMB/CIFS\)](#)

[Set up an Apache web server](#)

[Network boot your Raspberry Pi](#)

[Network boot using IPv6](#)

[Camera software](#)

[AI software](#)

[Raspberry Pi computer hardware](#)

[Keyboard computers](#)

[Compute Module hardware](#)

[Processors](#)

[I/O controllers](#)

Remote access

Introduction to remote access

[Edit this on GitHub](#)

Sometimes you need to access a Raspberry Pi without connecting it to a monitor, keyboard, and mouse. Perhaps the Raspberry Pi is embedded in a robot or mounted in an inconvenient location. Or maybe you don't have a spare monitor.

Remote control over the local network

To remotely control your Raspberry Pi from another device on your local network, use one of the following services:

- [SSH](#)
- [VNC](#)
- [Raspberry Pi Connect](#)

SSH (Secure Shell) provides secure access to a terminal session on your Raspberry Pi. VNC (Virtual Network Computing) provides secure access to a desktop screen share on your Raspberry Pi. All you need is another computer, a local network, and the local [IP address](#) of your Raspberry Pi. Raspberry Pi Connect shares your Raspberry Pi's screen securely with no need to determine your local IP address.

Share files between devices over the local network

Services like [NFS](#) (Network File System), [SCP](#) (Secure Copy Protocol), [Samba](#), and [rsync](#) enable you to share files between devices on the local network without directly controlling the remote device. These services can be useful when you need to access data stored on one device from another device.

Remote control over the Internet

To remotely control your Raspberry Pi from any device connected to the Internet, you can:

- Expose [SSH](#) or [VNC](#) on your Raspberry Pi over the open internet, within a VPN, or using an external service like RealVNC's cloud [VNC Viewer](#).
- Use [Raspberry Pi Connect](#), a free screen sharing and remote shell service provided by Raspberry Pi.

Find the IP address of your Raspberry Pi

[Edit this on GitHub](#)

Most methods of connecting to your Raspberry Pi from another machine require you to know the local IP address of your Raspberry Pi.

Any device connected to a Local Area Network is assigned an IP address. In order to connect to your Raspberry Pi from another machine using [SSH](#) or [VNC](#), you need to know the Raspberry Pi's IP address. This is easy if you have a display connected, and there are a number of methods for finding it remotely from another machine on the network.

To find the local IP address of your Raspberry Pi, use one of the following methods.

Desktop

On this page

[Introduction to remote access](#)

[Find the IP address of Raspberry Pi](#)

[Access a remote terminal with SSH](#)

[Screen share with VNC](#)

[Remote access with Raspberry Pi Connect](#)

[Share files with SCP](#)

[Synchronise folders between computers with rsync](#)

[Network File System \(Samba \(SMB/CIFS\)\)](#)

[Set up an Apache web server](#)

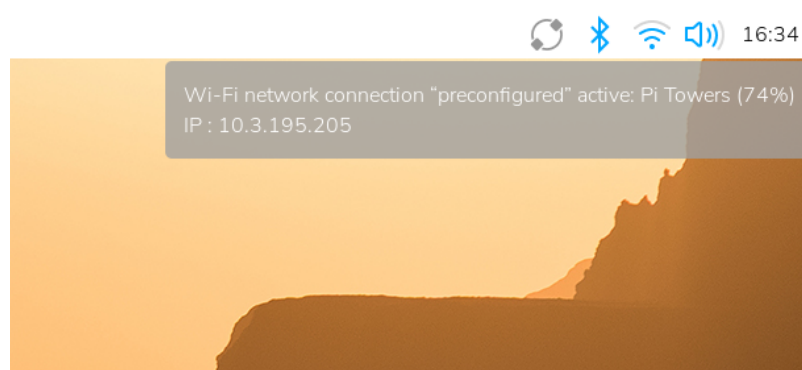
[Network boot your Raspberry Pi](#)

[Network boot using IP](#)

Software sources

Legacy config.txt
options

Hover over the network icon in the system tray, and a tooltip will appear. This tooltip displays the name of the network you're currently connected to and your IP address.



Command line

Run the following command to output your local IP address to the command line:

```
$ hostname -I
```

Boot output

If you use a display with your Raspberry Pi and you boot to the command line instead of the desktop, the boot sequence includes your IP address as one of the last few output messages before your login prompt.

Network Manager

You can use the built-in Network Manager CLI (`nmcli`) to access details about your network. Run the following command:

```
$ nmcli device show
```

You should see output similar to the following:

```
GENERAL.DEVICE: wlan0
GENERAL.TYPE: wifi
GENERAL.HWADDR: D0:3B:FF:41:AB:8A
GENERAL.MTU: 1500
GENERAL.STATE: 100 (connected)
GENERAL.CONNECTION: exampleNetworkName
GENERAL.CON-PATH: /org/freedesktop/NetworkManager/Acti
veConnection/2
IP4.ADDRESS[1]: 192.168.1.42/24
IP4.GATEWAY: 192.168.1.1
IP4.ROUTE[1]: dst = 192.168.1.0/24, nh = 0.0.0.0,
mt = 600
IP4.ROUTE[2]: dst = 0.0.0.0/0, nh = 192.168.1.1, m
t = 600
IP4.DNS[1]: 192.168.1.3
IP6.ADDRESS[1]: ab80::11ab:b1fc:bb7e:a8a5/64
IP6.GATEWAY: --
IP6.ROUTE[1]: dst = ab80::/64, nh = ::, mt = 1024

GENERAL.DEVICE: lo
GENERAL.TYPE: loopback
GENERAL.HWADDR: 00:00:00:00:00:00
GENERAL.MTU: 65536
GENERAL.STATE: 100 (connected (externally))
GENERAL.CONNECTION: lo
GENERAL.CON-PATH: /org/freedesktop/NetworkManager/Acti
veConnection/1
IP4.ADDRESS[1]: 127.0.0.1/8
IP4.GATEWAY: --
IP6.ADDRESS[1]: ::1/128
IP6.GATEWAY: --

GENERAL.DEVICE: p2p-dev-wlan0
GENERAL.TYPE: wifi-p2p
GENERAL.HWADDR: (unknown)
GENERAL.MTU: 0
GENERAL.STATE: 30 (disconnected)
GENERAL.CONNECTION: --
GENERAL.CON-PATH: --

GENERAL.DEVICE: eth0
GENERAL.TYPE: ethernet
GENERAL.HWADDR: D0:3B:FF:41:AB:89
```

```

GENERAL.MTU: 1500
GENERAL.STATE: 20 (unavailable)
GENERAL.CONNECTION: --
GENERAL.CON-PATH: --
WIRED-PROPERTIES.CARRIER: off
IP4.GATEWAY: --
IP6.GATEWAY: --

```

This command outputs information about the various network interfaces accessible on your Raspberry Pi. Check the `GENERAL.TYPE` row to see which kind of network interface each block describes. For example, "ethernet" is the Ethernet port on your device, and "wifi" refers to the Wi-Fi chip built into some devices. You'll look at different blocks of output to find your IP address depending on the way your device accesses the internet:

- if your device connects to the internet using Wi-Fi, check the "wifi" block
- if your device connects to the internet using the Ethernet port, check the "ethernet" block

When you identify the correct network interface block, look for a field named `IP4.ADDRESS[1]` for an IPv4 address or `IP6.ADDRESS[1]` for an IPv6 address. You can ignore the trailing slash and number (for example, `/24`) in those fields.

In the example above, the Raspberry Pi uses Wi-Fi to access the internet. Check the block where the `GENERAL.TYPE` field reads "wifi" to find the IP address. In this case, you can access this device using the IPv4 address in the `IP4.ADDRESS[1]` field:
192.168.1.42.

Resolve raspberrypi.local with mDNS

Raspberry Pi OS supports multicast DNS as part of the Avahi service.

If your device supports mDNS, you can reach your Raspberry Pi by using its hostname and the `.local` suffix. The default hostname on a fresh Raspberry Pi OS install is `raspberrypi`, so by default any Raspberry Pi running Raspberry Pi OS responds to:

```
$ ping raspberrypi.local
```

If the Raspberry Pi is reachable, `ping` will show its IP address:

```

PING raspberrypi.local (192.168.1.131): 56 data bytes
64 bytes from 192.168.1.131: icmp_seq=0 ttl=255 time=2.618 ms

```

TIP

If you change the system hostname of your Raspberry Pi using Control Centre, `raspi-config`, or `/etc/hostname`, Avahi updates the `.local` mDNS address. If you don't remember the hostname of your Raspberry Pi, you can install Avahi on another device, then use `avahi-browse` to browse all the hosts and services on your local network.

Check your router's list of devices

In a web browser, navigate to your router's IP address. Then, log in using your credentials.

TIP

Your router's IP address is often `http://192.168.1.1`, but not always. You may be able to find your router's address and credentials printed on a label on your router.

This will take you to a control panel. Browse to the list of connected devices or similar (all routers are different), and you should see some devices you recognise. Some devices are detected as PCs, tablets, phones, printers, etc. so you should recognise some and rule them out to figure out which is your Raspberry Pi.

TIP

If you connect your Raspberry Pi to your network with a wire, try filtering for wired devices in the list. There should be fewer devices to choose from.

Find devices with nmap

The Network Mapper command (`nmap`) is a free and open source tool for network discovery. It is available for Linux, macOS, and Windows.

- To install on **Linux**, install the `nmap` package, for example, `apt install nmap`.
- To install on **macOS** or **Windows**, see the [nmap.org download page](https://nmap.org/download).

To use `nmap` to scan the devices on your network, you need to know the subnet you are connected to. First, find the local IP address of the computer you're using:

- On **Linux**, type `hostname -I` into a terminal window
- On **macOS**, go to **System Settings > Network**, select your active network connection, then click the **Details...** button
- On **Windows**, go to the Control Panel, then under **Network and Sharing Center**, click **View network connections**, select your active network connection and click **View status of this connection**

Next, scan the whole **subnet** for other devices. Most local networks use IPv4, which uses four numbers with values between 1 and 255 for each IP address. Devices on your subnet all use the same first three numbers. For example, if your IP address is 192.168.1.5, other devices will use addresses like 192.168.1.2, 192.168.1.6 and 192.168.1.200. To scan this subnet with `nmap`, pass the string 192.168.1.0/24, which covers the subnet range 192.168.1.0 to 192.168.1.255. Use the `-sn` flag to run a **ping scan** on the entire subnet range:

```
$ sudo nmap -sn 192.168.1.0/24
```

TIP

This may take up to a minute depending on your local network speed.

A ping scan queries all IP addresses in the range for a response. For each device that responds to the ping, the output shows the hostname and IP address as follows:

```
Starting Nmap 6.40 ( http://nmap.org ) at 2014-03-10 12:46 GMT
Nmap scan report for hpprinter (192.168.1.2)
Host is up (0.00044s latency).
Nmap scan report for Gordons-MBP (192.168.1.4)
Host is up (0.0010s latency).
Nmap scan report for ubuntu (192.168.1.5)
Host is up (0.0010s latency).
Nmap scan report for raspberrypi (192.168.1.8)
Host is up (0.0030s latency).
Nmap done: 256 IP addresses (4 hosts up) scanned in 2.41 seconds
```

The output above shows a device with hostname `raspberrypi` has IP address 192.168.1.8.

Find devices with a smartphone app

The Fing app is a free network scanner for smartphones. It is available for [Android](#) and [iOS](#).

1. Connect your phone to the same network as your Raspberry Pi.
2. When you open the Fing app, touch the refresh button in the upper right-hand corner of the screen.
3. After a few seconds, you should see a list with all the devices connected to your network.
4. Scroll down to the entry with the manufacturer "Raspberry Pi". The IP address appears in the bottom left corner, and the MAC address in the bottom right corner of the entry.

Access a remote terminal with SSH

[Edit this on GitHub](#)

You can access the terminal of a Raspberry Pi remotely from another computer on the same network using the **Secure SHell** (SSH) protocol.

Enable the SSH server

By default, Raspberry Pi OS disables the SSH server. Enable SSH in one of the following ways:

On the desktop	While flashing a fresh OS image	From the terminal	Manually
----------------	---------------------------------	-------------------	----------

1. From the **Preferences** menu, launch **Control Centre**.
2. Navigate to the **Interfaces** tab.
3. Select **Enabled** next to **SSH**.
4. Click **OK**.

Connect to an SSH server

Open a terminal window on your computer and enter the following command, replacing the `<ip address>` placeholder with the **IP address of the Raspberry Pi you're trying to connect to** and `<username>` with your username:

```
$ ssh <username>@<ip address>
```

When the connection works, you will see a security warning. Type `yes` to continue. You will only see this warning the first time you connect.

Enter your account password when prompted.

You should now see the Raspberry Pi command prompt:

```
<username>@<hostname> ~ $
```

You are now connected to the Raspberry Pi remotely, and can execute commands.

NOTE

If you receive a `connection timed out` error, you may have entered the wrong IP address for the Raspberry Pi. Check the **IP address of the Raspberry Pi**.

Forward X11 over SSH

NOTE

On macOS and Windows, you must install a third-party X server to use X11 forwarding.

X11 enables graphical applications over SSH. Pass the `-Y` flag to forward an X session over SSH:

```
$ ssh -Y <username>@<ip address>
```

When authenticated, you see the command line as usual. However, you can also open graphical windows that an X server can render for you. For example, type the following command to launch a **Geany** window:

```
$ geany &
```

Configure SSH without a password

To remotely access your Raspberry Pi without providing a password each time you connect, use an SSH keypair.

Preconfigure an OS image with Raspberry Pi Imager

When configuring an operating system image with Raspberry Pi Imager, you can preconfigure SSH keys to use an existing key on the computer where you run Imager.

1. Follow the [install using Imager](#) guide to configure your boot image.
2. On the **Remote Access** tab, toggle the **Enable SSH** switch to the active position.
3. Select **Use public key authentication**.
 - If you already have an SSH public key stored in `~/.ssh/id_rsa.pub`, Imager automatically uses that public key to prefill the text box.
 - Otherwise, browse to the location of your public key.
4. Select **Next** to proceed with the imaging process.

Manually configure an SSH key

If you already have an installation of Raspberry Pi OS, you can update your existing configuration to use SSH key authentication.

Check for existing SSH public keys

To check for an existing SSH public key on the computer you use to remotely connect to the Raspberry Pi, run the following command:

```
$ ls ~/.ssh
```

If you see files named `id_ed25519.pub`, `id_rsa.pub`, or `id_dsa.pub`, you already have an SSH key. Skip SSH keypair generation and proceed to [add the SSH key to your list of SSH identities](#).

Generate new SSH keypair

TIP

This guide provides instructions to generate a new RSA key. For additional security, you can instead generate a **Ed25519** key. Pass `-t ed25519` to `ssh-keygen` and replace `rsa` with `ed25519` when referencing your public and private key file names to use an Ed25519 key.

To generate a new SSH keypair, enter the following command:

```
$ ssh-keygen
```

When asked where to save the key, press **Enter** to use the default location, `~/.ssh/id_rsa`.

When asked for an optional keyphrase, press **Enter** to use no keyphrase.

Run the following command to check the contents of the `.ssh` directory:

```
$ ls ~/.ssh
```

You should see the files `id_rsa` and `id_rsa.pub`:

```
authorized_keys  id_rsa  id_rsa.pub  known_hosts
```

The `id_rsa` file contains your private key. Keep this secure on the computer you use to remotely connect to the Raspberry Pi.

The `id_rsa.pub` file contains your public key. You will share this key with your Raspberry Pi. When you connect with the Raspberry Pi remotely, it will use this key to verify your identity.

Add the SSH key to your list of SSH identities

Start the SSH agent:

```
$ eval "$(ssh-agent -s)"
```

Next, add your key identities to `ssh-agent` with the following command:

```
$ ssh-add ~/.ssh/id_rsa
```

Copy a public key to your Raspberry Pi

On the computer you use to remotely connect to the Raspberry Pi, use the following command to securely copy your public key to the Raspberry Pi:

```
$ ssh-copy-id <username>@<ip address>
```

When prompted, enter the password for your user account on the Raspberry Pi. You can now connect to your Raspberry Pi without entering a password.

Manually copy a public key to your Raspberry Pi

If your operating system does not support `ssh-copy-id`, you can instead copy your public key with `scp`.

First, *on your Raspberry Pi*, create the directory where Linux expects to find keys:

```
$ mkdir .ssh
```

Then, configure the proper permissions for the `.ssh` directory:

```
$ chmod 700 .ssh
```

On your usual computer, use `scp` to copy your public key to a file named `.ssh/authorized_keys` on your Raspberry Pi:

```
$ scp .ssh/id_rsa.pub <username>@<ip address>:.ssh/authorized_keys
```

TIP

The command above assumes you have never before authorized any keys to access your Raspberry Pi. If you have previously added at least one key, you should instead add a new line containing the public key to the end of the `authorized_keys` file to preserve your existing keys.

When prompted, enter the password for your user account on the Raspberry Pi.

Then, *on your Raspberry Pi*, configure permissions for the `authorized_keys` file:

```
$ chmod 644 .ssh/authorized_keys
```

You can now connect to your Raspberry Pi without entering a password.

Screen share with VNC

[Edit this on GitHub](#)

Sometimes it is not convenient to physically work with a device. Virtual Network Computing (VNC) allows you to control the desktop of one device from another.

VNC relies upon a client and a server. The client runs on a device you can physically interact with, such as a personal laptop, desktop, tablet, or phone. The server runs on your Raspberry Pi. When you use VNC, the client transmits keyboard and mouse events to the server. The server executes those events on your Raspberry Pi, and returns screen updates to the client.

The VNC client displays the desktop of your Raspberry Pi in a window. You can interact with the desktop as though you were working on the Raspberry Pi itself.

Raspberry Pi OS includes `wayvnc`. This provides a VNC server that you can enable in your device preferences.

Before you can use VNC on your Raspberry Pi, you must enable the VNC server.

Enable the VNC server

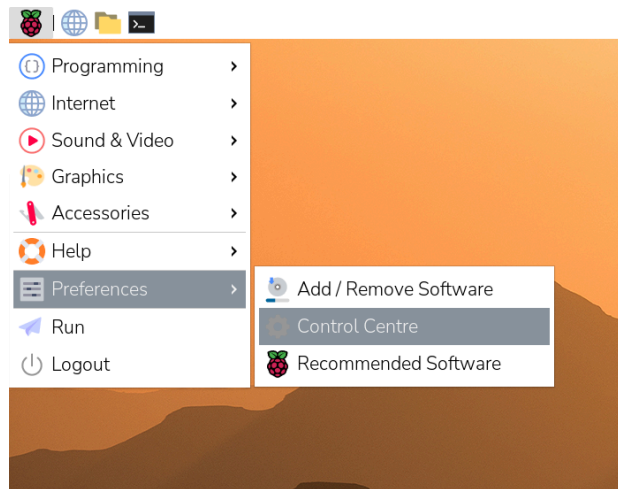
Raspberry Pi OS supports enabling the VNC server both graphically and at the command line.

TIP

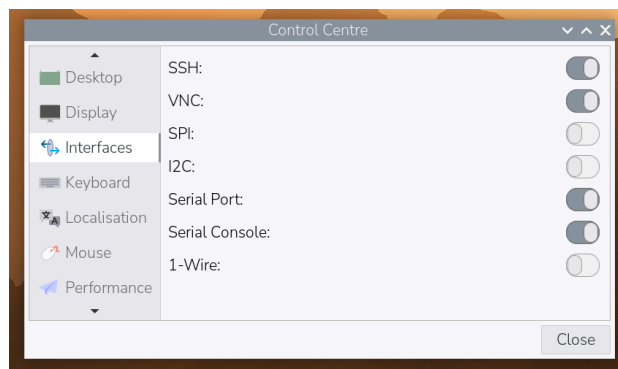
When enabled, you can access your WayVNC configuration at `/etc/wayvnc/`.

Enable VNC Server Graphically

1. Boot into the graphical desktop on your Raspberry Pi.
2. Select the Raspberry Pi icon in the system tray of your desktop.
3. Select **Preferences > Control Centre** from the menu.



4. Navigate to the **Interfaces** tab.
5. Toggle the radio button next to **VNC** into the active position.



6. Select the **OK** button to save your configuration changes.

Enable the VNC server on the command line

Use **raspi-config** to enable the VNC server on the command line.

1. Open **raspi-config** with the following line:

```
$ sudo raspi-config
```
2. Navigate to **Interface Options**. Press Enter to select.
3. Select **VNC**. Press Enter to select.
4. Under **Would you like the VNC Server to be enabled?**, highlight <Yes> and press Enter.
5. Press Enter to return to the menu. Press Esc to exit **raspi-config**.

Connect to a VNC server

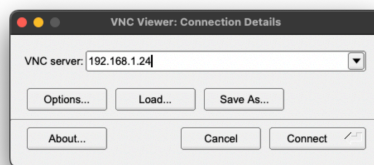
To connect to your Raspberry Pi, you'll need the following:

- your Raspberry Pi and the device running the VNC client, connected to the same network (for example, a home wireless network or VPN)

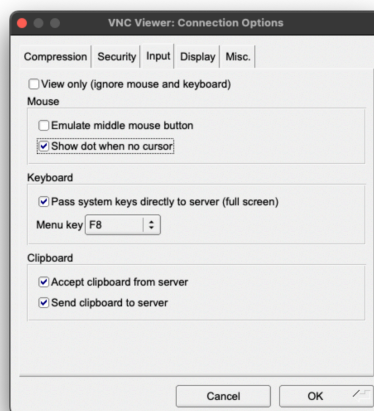
- the hostname or IP address of your Raspberry Pi
- a valid username and password combination for an account on your Raspberry Pi

If you don't know the IP address of your device, see [our instructions on finding your IP address](#).

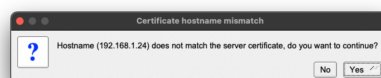
1. Download **TigerVNC**. You can install the latest version from the [Releases page of their GitHub repository](#). Click on the link in the latest release, and find the binary for your platform. Windows users should download an `exe`; macOS users should download the `dmg`; Linux users should install the `jar`.
2. On your client device, launch TigerVNC. On macOS and Windows, you can double-click the binary. On Linux, install java with `sudo apt install default-jre`, then run `java -jar VncViewer-<version>.jar`, replacing the `<version>` placeholder with the version you downloaded.
3. In the "VNC server" field, enter the IP address of your Raspberry Pi.



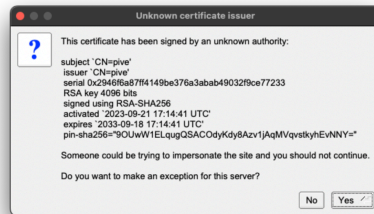
4. Click the "Options" button. Navigate to the "Input" tab. Check the box next to "Show dot when no cursor" to ensure that you can always see a cursor in TigerVNC.



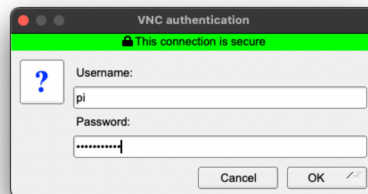
5. Click the "Connect" button to initiate a connection with the server.
 - If TigerVNC warns you that the "Hostname does not match the server certificate", click the "Yes" button to continue.



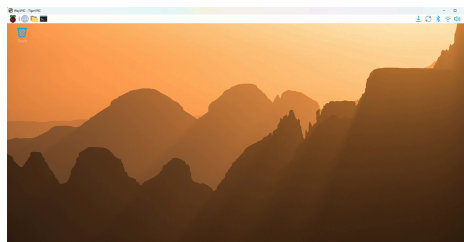
- If TigerVNC warns you that the "certificate has been signed by an unknown authority", click the "Yes" button to grant an exception for your Raspberry Pi.



6. When prompted for a username and password, enter your credentials.



7. Click the "OK" button to authenticate with the VNC server. If your credentials are correct, TigerVNC should open a window containing the desktop corresponding to your account on the Raspberry Pi. You should be able to use your mouse and keyboard to interact with the desktop and input text.



Remote access with Raspberry Pi Connect

[Edit this on GitHub](#)

You can access a Raspberry Pi remotely from a browser on another device using Raspberry Pi Connect. Connect handles configuration automatically, so you don't have to find your Raspberry Pi's local IP address, your network's public IP address, or modify your local network firewall to enable external access.

Connect includes the ability to screen share on Raspberry Pi models running the Wayland window server and remote shell (terminal) access on all Raspberry Pi models.

For more information, see [the Connect documentation](#).

Share files with SCP

[Edit this on GitHub](#)

Secure Copy Protocol (scp) sends files over SSH. You can use scp to copy files between your Raspberry Pi and another computer.

To use scp, [find your Raspberry Pi's IP address](#).

Copy files to your Raspberry Pi

To copy a file named `myfile.txt` from your personal computer to a user's home folder on your Raspberry Pi, run the following command from the directory containing `myfile.txt`, replacing the `<username>` placeholder with the username you use to log in to your Raspberry Pi and the `<pi_ip_address>` placeholder with your Raspberry Pi's IP address:

```
$ scp myfile.txt <username>@<pi_ip_address>:
```

To copy a file to a specific directory, append the directory path after the `:` in the `scp` command. Create the folder before you run `scp`, since `scp` won't create folders automatically. For instance, the following command copies a file named `myfile.txt` into the `project/` directory within a user's home folder:

```
$ scp myfile.txt <username>@<pi_ip_address>:project/
```

Copy files from your Raspberry Pi

To copy a file named `myfile.txt` from a user's home directory on a Raspberry Pi to the current directory on another computer, run the following command:

```
$ scp <username>@<pi_ip_address>:myfile.txt .
```

Copy multiple files with one command

To copy multiple files, list the file names in a single command, separated by spaces:

```
$ scp myfile.txt myfile2.txt <username>@<pi_ip_address>:
```

Alternatively, use a wildcard to copy all files matching a particular filter. The following command copies all files that end with `.txt`:

```
$ scp *.txt <username>@<pi_ip_address>:
```

The following command copies all files that start with `m`:

```
$ scp m* <username>@<pi_ip_address>:
```

The following command copies all files that start with `m` and end with `.txt`:

```
$ scp m*.txt <username>@<pi_ip_address>:
```

NOTE

To copy files with names that contain spaces, enclose the file name in quotes:

```
$ scp "my file.txt" <username>@<pi_ip_address>:
```

Copy a folder

To copy a folder and all of its contents, pass the folder name with the `-r` (recursive) flag:

```
$ scp -r project/ <username>@<pi_ip_address>:
```

Synchronise folders between computers with `rsync`

Edit this [on GitHub](#)

You can use `rsync` to synchronise folders between computers. For example, you could use `rsync` to transfer new pictures taken by your Raspberry Pi to your personal computer automatically.

Before you can configure `rsync`, determine values for the following:

- `<pi_ip_address>`: your Raspberry Pi's local IP address: see [Find your Raspberry Pi's IP address](#) for more information

- `<pi_username>`: the username you use to log into your Raspberry Pi
- `<pi_folder_name>`: the name of the folder you want to copy files from on your Raspberry Pi
- `<pc_folder_name>`: the name of the folder you would like to synchronise on your personal computer

To configure `rsync` to synchronise files, complete the following steps on your personal computer, replacing placeholders in the commands with the values you determined above:

1. Create the folder you would like to synchronise to:

```
$ mkdir <pc_folder_name>
```

2. Synchronise files to the folder with `rsync`:

```
$ rsync -avz -e ssh <pi_username>@<pi_ip_address>:<pi_folder_name>/ <pc_folder_name>/
```

This command copies all files from the selected folder on your Raspberry Pi to the selected folder on your personal computer. If you run the command multiple times, `rsync` keeps track of the files you have already downloaded and skips them. If you delete or modify an already synchronised file on the Raspberry Pi, `rsync` updates the files on your personal computer accordingly.

Network File System (NFS)

[Edit this on GitHub](#)

Network File System (NFS) allows you to share a directory located on one networked computer with other computers or devices on the same network. The computer where the directory is located is called the **server**, and computers or devices connecting to that server are called clients. Clients usually `mount` the shared directory to make it a part of their own directory structure. The shared directory is an example of a shared resource or network share.

NFS is a popular way to create a simple NAS (Network-attached storage) in a Linux/Unix environment.

An NFS is perhaps best suited to more permanent network-mounted directories, such as `/home` directories or regularly-accessed shared resources. If you want a network share that guest users can easily connect to, Samba is better suited to the task. Tools to temporarily mount and detach from Samba shares are more readily available across operating systems.

Before deploying an NFS, you should be familiar with:

- Linux file and directory permissions
- mounting and unmounting filesystems

Set up a basic NFS server

Install the packages required using the command below:

```
$ sudo apt install nfs-kernel-server
```

For easier maintenance, we will isolate all NFS exports in single directory, into which the real directories will be mounted with the `--bind` option.

Suppose we want to export our users' home directories, which are in `/home/users`. First we create the export filesystem:

```
$ sudo mkdir -p /export/users
```

TIP

If you plan to configure LDAP/NIS authentication, skip the `chmod` step below.

Grant `/export` and `/export/users` read, write, and execute permissions (777) so you can access the NFS share from the client without LDAP/NIS authentication:

```
$ chmod -R 777 /export
```

Next, mount the real `users` directory with:

```
$ sudo mount --bind /home/users /export/users
```

To save us from retyping this after every reboot, we add the following line to `/etc/fstab`:

```
/home/users    /export/users  none    bind  0 0
```

There are three configuration files that relate to an NFS server:

1. `/etc/default/nfs-kernel-server`
2. `/etc/default/nfs-common`
3. `/etc/exports`

The only important option in `/etc/default/nfs-kernel-server` for now is `NEED_SVCSSD`. It is set to "no" by default, which is fine, because we are not activating NFSv4 security this time.

In order for the ID names to be automatically mapped, the file `/etc/idmapd.conf` must exist on both the client and the server with the same contents and with the correct domain names. Furthermore, this file should have the following lines in the `Mapping` section:

```
[Mapping]
Nobody-User = nobody
Nobody-Group = nogroup
```

However, note that the client may have different requirements for the `Nobody-User` and `Nobody-Group`. For example, on RedHat variants, it is `nfsnobody` for both. If you're not sure, check via the following commands to see if `nobody` and `nogroup` are there:

```
$ cat /etc/passwd
$ cat /etc/group
```

This way, server and client do not need the users to share same UID/GUID. For those who use LDAP-based authentication, add the following lines to the `idmapd.conf` of your clients:

```
[Translation]
Method = nsswitch
```

This will cause `idmapd` to know to look at `nsswitch.conf` to determine where it should look for credential information. If you have LDAP authentication already working, `nsswitch` shouldn't require further explanation.

To export our directories to a local network `192.168.1.0/24`, add the following two lines to `/etc/exports`:

```
/export      192.168.1.0/24(rw,fsid=0,insecure,no_subtree_check,async)
/export/users 192.168.1.0/24(rw,nohide,insecure,no_subtree_check,async)
```

Portmap lockdown (optional)

The files on your NFS are open to anyone on the network. As a security measure, you can restrict access to specified clients.

Add the following line to `/etc/hosts.deny`:

```
rpcbind mountd nfsd statd lockd rquotad : ALL
```

By blocking all clients first, only clients in `/etc/hosts.allow` (added below) will be allowed to access the server.

Now add the following line to `/etc/hosts.allow`:

```
rpcbind mountd nfsd statd lockd rquotad : <list of IPv4s>
```

where `<list of IPv4s>` is a list of the IP addresses of the server and all clients. (These have to be IP addresses because of a limitation in `rpcbind`, which doesn't like hostnames.) Note that if you have NIS set up, you can just add these to the same line.

Please ensure that the list of authorised IP addresses includes the `localhost` address (`127.0.0.1`), as the startup scripts in recent versions of Ubuntu use the `rpcinfo` command to discover NFSv3 support, and this will be disabled if `localhost` is unable to connect.

Finally, to make your changes take effect, restart the service:

```
$ sudo systemctl restart nfs-kernel-server
```

Configure an NFS client

Now that your server is running, you need to set up any clients to be able to access it. To start, install the required packages:

```
$ sudo apt install nfs-common
```

On the client, we can mount the complete export tree with one command:

```
$ mount -t nfs -o proto=tcp,port=2049 <nfs-server-IP>:/ /mnt
```

You can also specify the NFS server hostname instead of its IP address, but in this case you need to ensure that the hostname can be resolved to an IP on the client side. A robust way of ensuring that this will always resolve is to use the `/etc/hosts` file.

Note that `<nfs-server-IP>:/export` is not necessary in NFSv4, as it was in NFSv3. The root export `:/` defaults to export with `fsid=0`.

We can also mount an exported subtree with:

```
$ mount -t nfs -o proto=tcp,port=2049 <nfs-server-IP>:/users /home/users
```

To ensure this is mounted on every reboot, add the following line to `/etc/fstab`:

```
<nfs-server-IP>:/ /mnt nfs auto 0 0
```

If, after mounting, the entry in `/proc/mounts` appears as `<nfs-server-IP>://` (with two slashes), then you might need to specify two slashes in `/etc/fstab`, or else `umount` might complain that it cannot find the mount.

Portmap lockdown (optional)

Add the following line to `/etc/hosts.deny`:

```
rpcbind : ALL
```

By blocking all clients first, only clients in `/etc/hosts.allow` (added below) will be allowed to access the server.

Now add the following line to `/etc/hosts.allow`:

```
rpcbind : <NFS server IP address>
```

where `<NFS server IP address>` is the IP address of the server.

Configure a complex NFS server

NFS user permissions are based on user ID (UID). UIDs of any users on the client must match those on the server in order for the users to have access. The typical ways of doing this are:

- Manual password file synchronisation
- Use of LDAP
- Use of DNS
- Use of NIS

Note that you have to be careful on systems where the main user has root access: that user can change UIDs on the system to allow themselves access to anyone's files. This page assumes that the administrative team is the only group with root access and that they are all trusted. Anything else represents a more advanced configuration, and will not be addressed here.

Group permissions

A user's file access is determined by their membership of groups on the client, not on the server. However, there is an important limitation: a maximum of 16 groups are passed from the client to the server, and if a user is member of more than 16 groups on the client, some files or directories might be unexpectedly inaccessible.

DNS (optional, only if using DNS)

Add any client name and IP addresses to `/etc/hosts`. (The IP address of the server should already be there.) This ensures that NFS will still work even if DNS goes down. Alternatively you can rely on DNS if you want – it's up to you.

NIS (optional, only if using NIS)

This applies to clients using NIS. Otherwise you can't use netgroups, and should specify individual IPs or hostnames in `/etc/exports`. Read the BUGS section in `man netgroup` for more information.

First, edit `/etc/netgroup` and add a line to classify your clients (this step is not necessary, but is for convenience):

```
myclients (client1,,) (client2,,) ...
```

where `myclients` is the netgroup name.

Next run this command to rebuild the NIS database:

```
$ sudo make -C /var/yp
```

The filename `yp` refers to Yellow Pages, the former name of NIS.

Portmap lockdown (optional)

Add the following line to `/etc/hosts.deny`:

```
rpcbind mountd nfsd statd lockd rquotad : ALL
```

By blocking all clients first, only clients in `/etc/hosts.allow` (added below) will be allowed to access the server.

Consider adding the following line to `/etc/hosts.allow`:

```
rpcbind mountd nfsd statd lockd rquotad : <list of IPs>
```

where <list of IPs> is a list of the IP addresses of the server and all clients. These have to be IP addresses because of a limitation in `rpcbind`. Note that if you have NIS set up, you can just add these to the same line.

Package installation and configuration

Install the necessary packages:

```
$ sudo apt install rpcbind nfs-kernel-server
```

Edit `/etc/exports` and add the shares:

```
/home @myclients(rw,sync,no_subtree_check)
/usr/local @myclients(rw,sync,no_subtree_check)
```

The example above shares `/home` and `/usr/local` to all clients in the `myclients` netgroup.

```
/home 192.168.0.10(rw,sync,no_subtree_check) 192.168.0.11(rw,sync,no_subtree_check)
/usr/local 192.168.0.10(rw,sync,no_subtree_check) 192.168.0.11(rw,sync,no_subtree_check)
```

The example above shares `/home` and `/usr/local` to two clients with static IP addresses. If you want instead to allow access to all clients in the private network falling within a designated IP address range, consider the following:

```
/home 192.168.0.0/255.255.255.0(rw,sync,no_subtree_check)
/usr/local 192.168.0.0/255.255.255.0(rw,sync,no_subtree_check)
```

Here, `rw` makes the share read/write, and `sync` requires the server to only reply to requests after any changes have been flushed to disk. This is the safest option; `async` is faster, but dangerous. It is strongly recommended that you read `man exports` if you are considering other options.

After setting up `/etc/exports`, export the shares:

```
$ sudo exportfs -ra
```

You'll want to run this command whenever `/etc/exports` is modified.

Restart services

Restart `rpcbind` and NFS for the changes to take effect:

```
$ sudo systemctl restart rpcbind
$ sudo systemctl restart nfs-kernel-server
```

Security items to consider

Aside from the UID issues discussed above, it should be noted that an attacker could potentially masquerade as a machine that is allowed to map the share, which allows them to create arbitrary UIDs to access your files. One potential solution to this is IPSec. You can set up all your domain members to talk to each other only over IPSec, which will effectively authenticate that your client is who it says it is.

IPSec works by encrypting traffic to the server with the server's public key, and the server sends back all replies encrypted with the client's public key. The traffic is decrypted with the respective private keys. If the client doesn't have the keys that it is supposed to have, it can't send or receive data.

An alternative to IPSec is physically separate networks. This requires a separate network switch and separate Ethernet cards, and physical security of that network.

Troubleshooting

Mounting an NFS share inside an encrypted home directory will only work after you are successfully logged in and your home is decrypted. This means that using `/etc/fstab` to mount NFS shares on boot will not work, because your home has not

been decrypted at the time of mounting. There is a simple way around this using symbolic links:

1. Create an alternative directory to mount the NFS shares in:

```
$ sudo mkdir /nfs
$ sudo mkdir /nfs/music
```

1. Edit `/etc/fstab` to mount the NFS share into that directory instead:

```
nfsServer:music    /nfs/music    nfs    auto    0 0
```

1. Create a symbolic link inside your home, pointing to the actual mount location. For example, and in this case deleting the `Music` directory already existing there first:

```
$ rmdir /home/user/Music
$ ln -s /nfs/music/ /home/user/Music
```

Samba (SMB/CIFS)

Edit this [on GitHub](#)

Samba is a free software reimplementation of the **Server Message Block** (SMB) networking protocol. With Samba, you can share folders between Windows, macOS, and Linux machines.

Install Samba on your Raspberry Pi

By default, Raspberry Pi OS does not include Samba. To install Samba on your Raspberry Pi, run the following command, which installs all the dependencies you need to run a Samba server or client:

```
$ sudo apt update
$ sudo apt install samba samba-common-bin smbclient cifs-utils
```

Mount a folder shared from Windows

First, you need to share a folder on your Windows device.

Turn on sharing

1. Right click the system tray and select **Networking and Sharing Centre** from the menu.
2. Select **Change advanced sharing settings**.
3. Select **Turn on network discovery**.
4. Select **Turn on file and printer sharing**.
5. Click the **Save** button to save your changes.

Share the folder

Follow these steps to share a folder from Windows:

1. Right click the folder you want to share and select **Properties**.
2. Select the **Sharing** tab.
3. Click the **Advanced Sharing** button.
4. Select **Share this folder**; by default, Windows uses the folder name as the share name.
5. Click the **Permissions** button.
6. Configure the **Everyone** and **Full Control** permissions.
7. Click the **OK** button to leave the **Permissions** page.
8. Click the **OK** button again to leave the **Advanced Sharing** page.

9. Select the **Security** tab.
10. Configure the **Everyone** and **Full Control** permissions.
11. Click the **OK** button.

The folder should now be shared. You can modify shared folder permissions by changing permissions on both the **Permissions** and **Security** pages.

Windows 10 Sharing Wizard

On Windows 10 there is a Sharing Wizard that helps with some of these steps.

1. Run the **Computer Management** application from the Start Bar.
2. Select **Shared Folders > Shares**.
3. Right click and select **New Share** to begin the Sharing Wizard.
4. Click the **Next** button.
5. Select the folder you wish to share, then click the **Next** button.
6. Click **Next** to use the sharing defaults or select **Custom** and set the required permissions.
7. Click the **OK** button.
8. Click the **Finish** button to share the folder.

Mount the folder on the Raspberry Pi

Mounting in Linux is the process of attaching a folder to a location, so firstly we need that location.

```
$ mkdir windowshare
```

Now, we need to mount the remote folder to that location. The remote folder is the host name or IP address of the Windows PC, and the share name used when sharing it. We also need to provide the Windows username that will be used to access the remote machine. Don't forget to replace the <username> placeholder with your Raspberry Pi OS username.

```
$ sudo mount.cifs //<hostname or IP address>/<shared windows folder> /home/<username>/windowshare -o user=<name>
```

You should now be able to view the content of the Windows share on your Raspberry Pi.

```
$ ls windowshare/
```

"Host is down" error

This error occurs when SMB protocol versions don't match and the Linux Samba client returns a misleading error message. By default, Raspberry Pi OS uses versions 2.1 and later, compatible with Windows 7 and later. Older devices, including some NAS devices, might require version 1.0. To fix this error, append a version entry (for example, ,vers=1.0) to your mount command:

```
$ sudo mount.cifs //IP/share /mnt/point -o user=<uname>,vers=1.0
```

You might need to try different versions to match up with the server version. Possible values are:

Version	Description
1.0	Classic CIFS/SMBv1 protocol
2.0	The SMBv2.002 protocol. Windows Vista Service Pack 1, and Windows Server 2008
2.1	The SMBv2.1 protocol. Microsoft Windows 7 and Windows Server 2008R2

Version	Description
3.0	The SMBv3.0 protocol. Microsoft Windows 8 and Windows Server 2012
3.02	The SMBv3.0.2 protocol. Microsoft Windows 8.1 and Windows Server 2012R2
3.11	The SMBv3.1.1 protocol. Microsoft Windows 10 and Windows Server 2016
3	The SMBv3.0 protocol version and above

Sharing a Folder from your Raspberry Pi

Firstly, create a folder to share. This example creates a folder called `shared` in the home folder of the current user:

```
$ cd ~
$ mkdir shared
$ chmod 0740 shared
```

Now we need to tell Samba about your default user account when accessing that folder. When prompted, enter your password, replacing the `<username>` placeholder with the username of your primary user account:

```
$ sudo smbpasswd -a <username>
```

Now we need to tell Samba to share this folder, using the Samba configuration file.

```
sudo nano /etc/samba/smb.conf
```

At the end of the file, add the following to share the folder, giving the remote user read/write permissions. Replace the `<username>` placeholder with the username of the primary user account on your Raspberry Pi:

```
[share]
  path = /home/<username>/shared
  read only = no
  public = yes
  writable = yes
```

In the same file, find the `workgroup` line, and if necessary, change it to the name of the workgroup of your local Windows network.

```
workgroup = <your workgroup name here>
```

The shared folder should now appear to Windows or macOS devices on the network. Enter your Raspberry Pi username and password to mount the folder.

Set up an Apache web server

[Edit this on GitHub](#)

Apache is a popular web server application you can install on the Raspberry Pi to allow it to serve web pages.

On its own, Apache can serve HTML files over HTTP, and with additional modules can serve dynamic web pages using scripting languages such as PHP.

Install Apache

First, update the available packages by typing the following command into the Terminal:

```
sudo apt update
```

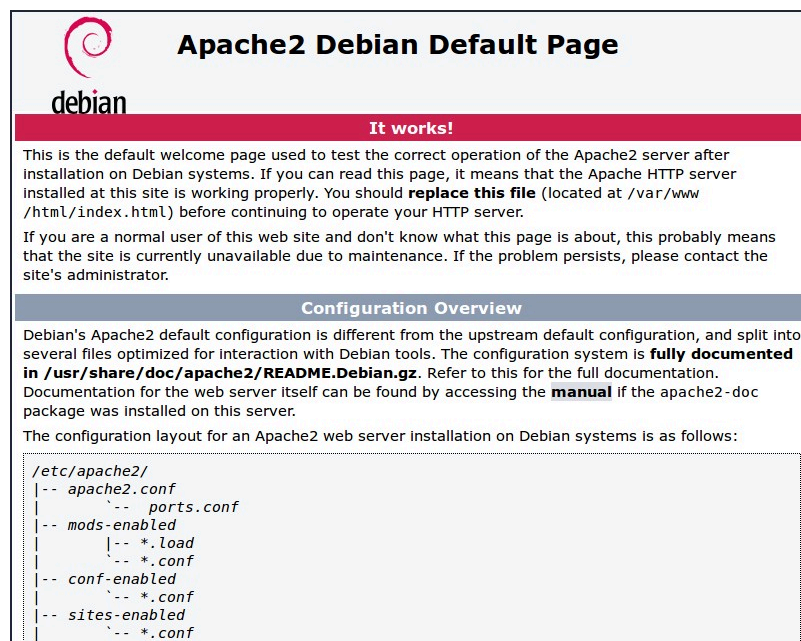
Then, install the `apache2` package with this command:

```
sudo apt install apache2 -y
```

Test the web server

By default, Apache puts a test HTML file in the web folder. This default web page is served when you browse to `http://localhost/` on the Raspberry Pi itself, or `http://192.168.1.10` (whatever the Raspberry Pi's IP address is) from another computer on the network. To find the Raspberry Pi's IP address, type `hostname -I` at the command line (or read more about finding your [IP address](#)).

Browse to the default web page either on the Raspberry Pi or from another computer on the network and you should see the following:



This means you have Apache working!

Change the default web page

This default web page is just an HTML file on the filesystem. It is located at `/var/www/html/index.html`.

Navigate to this directory in a terminal window and have a look at what's inside:

```
cd /var/www/html
ls -al
```

This will show you:

```
total 12
drwxr-xr-x  2 root root 4096 Jan  8 01:29 .
drwxr-xr-x 12 root root 4096 Jan  8 01:28 ..
-rw-r--r--  1 root root  177 Jan  8 01:29 index.html
```

This shows that by default there is one file in `/var/www/html/` called `index.html` and it is owned by the `root` user (as is the enclosing folder). In order to edit the file, you need to change its ownership to your own username. Change the owner of the file using the following command, replacing the `<username>` placeholder with the username of your primary user account:

```
$ sudo chown <username>: index.html
```

You can now try editing this file and then refreshing the browser to see the web page change. If you know HTML you can put your own HTML files and other assets in this directory and serve them as a website on your local network.

Install PHP for Apache

To allow your Apache server to process PHP files, you'll need to install the latest version of PHP and the PHP module for Apache. Type the following command to install these:

```
sudo apt install php libapache2-mod-php -y
```

Now remove the `index.html` file:

```
sudo rm index.html
```

and create the file `index.php`:

```
sudo nano index.php
```

Put some PHP content in it:

```
<?php echo "hello world"; ?>
```

Now save and refresh your browser. You should see "hello world". This is not dynamic but still served by PHP. Try something dynamic:

```
<?php echo date('Y-m-d H:i:s'); ?>
```

or show your PHP info:

```
<?php phpinfo(); ?>
```

Network boot your Raspberry Pi

[Edit this on GitHub](#)

You can set up a DHCP/TFTP server which allows you to boot a Raspberry Pi 3 or later from the network.

The instructions assume that you have an existing home network, and that you want to use a Raspberry Pi for the **server**. You will also need an additional Raspberry Pi 3 or 4 as a **client** to be booted. Only one SD Card is needed because the client will be booted from the server after the initial client configuration.

NOTE

Due to the huge range of networking devices and routers available, we can't guarantee that network booting will work with any device. We have had reports that, if you cannot get network booting to work, disabling STP frames on your network may help.

Configure a network boot client

Raspberry Pi 3 Model B

NOTE

This section only applies to the Raspberry Pi 3 Model B, as network boot is enabled on the Raspberry Pi 3 Model B+ at the factory.

Before the Raspberry Pi 3 Model B can network boot it needs to be booted from an SD card with a config option to enable USB boot mode. This sets a bit in the OTP (One Time Programmable) memory in the Raspberry Pi SoC that enables network booting. When this is done, the Raspberry Pi 3B attempts to boot from USB, and from the network, if it can't boot from the SD card.

Install Raspberry Pi OS Lite, or Raspberry Pi OS with desktop, on the SD card in the usual fashion. Next, enable USB boot mode with the following command:

```
$ echo program_usb_boot_mode=1 | sudo tee -a /boot/firmware/config.txt
```

This adds `program_usb_boot_mode=1` to the end of `/boot/firmware/config.txt`. Reboot the Raspberry Pi with `sudo reboot`. After the client Raspberry Pi has rebooted, check that the OTP has been programmed with:

```
$ vcgencmd otp_dump | grep 17:  
17:3020000a
```

Ensure the output `0x3020000a` is correct.

The client configuration is almost done. As a final step, disable USB booting. Run the following command:

```
$ sudo nano /boot/firmware/config.txt
```

Remove the line that contains the text `program_usb_boot_mode=1`. Finally, shut the client Raspberry Pi down with `sudo poweroff`.

Raspberry Pi 4 Model B

Network boot can be enabled on the Raspberry Pi 4 using the `raspi-config` tool. First, run `raspi-config` as follows:

```
$ sudo raspi-config
```

Within `raspi-config`, choose `Advanced Options`, then `Boot Order`, then `Network Boot`. You must then reboot the device for the change to the boot order to be programmed into the bootloader EEPROM. After the Raspberry Pi reboots, check that the boot order is now `0xf21`:

```
$ vcgencmd bootloader_config
```

For further details of configuring the Raspberry Pi 4 bootloader, see [Raspberry Pi Bootloader Configuration](#).

Ethernet MAC address

Before configuring network boot, make a note of the serial number and mac address so that the board can be identified by the TFTP/DHCP server.

On Raspberry Pi 4 and later flagship models, the MAC address is programmed at manufacture and there is no link between the MAC address and serial number. Both the MAC address and serial numbers are displayed on the bootloader **HDMI diagnostics** screen.

To find the Ethernet MAC address, run the following command:

```
$ ethtool -P eth0
```

To find the serial number, run the following command:

```
$ grep Serial /proc/cpuinfo | cut -d ' ' -f 2 | cut -c 9-16
```

Configure a network boot server

Plug the SD card into the server Raspberry Pi, and then boot the server. The client Raspberry Pi will need a root file system to boot from: we will use a copy of the server's root filesystem and place it in `/nfs/client1`:

```
$ sudo mkdir -p /nfs/client1
$ sudo apt install rsync
$ sudo rsync -xa --progress --exclude /nfs / /nfs/client1
```

Regenerate SSH host keys on the client filesystem by chrooting into it:

```
$ cd /nfs/client1
$ sudo mount --bind /dev dev
$ sudo mount --bind /sys sys
$ sudo mount --bind /proc proc
$ sudo chroot .
$ rm /etc/ssh/ssh_host_*
$ dpkg-reconfigure openssh-server
$ exit
$ sudo umount dev sys proc
```

Find the settings of your local network. You need to find the address of your router (or gateway), which can be done with:

```
$ ip route | awk '/default/ {print $3}'
```

Then run:

```
$ ip -4 addr show dev eth0 | grep inet
```

You should see output similar to the following:

```
inet 10.42.0.211/24 brd 10.42.0.255 scope global eth0
```

The first address is the IP address of your server Raspberry Pi on the network, and the part after the slash is the network size. It is highly likely that yours will be a /24. Also note the `brd` (broadcast) address of the network. Note down the output of the previous command, which will contain the IP address of the Raspberry Pi and the broadcast address of the network.

Finally, note down the address of your DNS server, which is the same address as your gateway. You can find this with:

```
$ cat /etc/resolv.conf
```

Configure a static network address on your server Raspberry Pi via the `systemd` networking, which works as the network handler and DHCP server.

To do that, you'll need to create a `10-eth0.netdev` and a `11-eth0.network` like so:

```
$ sudo nano /etc/systemd/network/10-eth0.netdev
```

Add the following lines:

```
[Match]
Name=eth0

[Network]
DHCP=no
```

Then create a network file:

```
$ sudo nano /etc/systemd/network/11-eth0.network
```

Add the following contents:

```
[Match]
Name=eth0

[Network]
Address=10.42.0.211/24
DNS=10.42.0.1

[Route]
Gateway=10.42.0.1
```

At this point, you will not have working DNS, so you will need to add the server you noted down before to `systemd/resolved.conf`. In this example, the gateway address is 10.42.0.1.

```
$ sudo nano /etc/systemd/resolved.conf
```

Uncomment the DNS line and add the DNS IP address there. Additionally, if you have a fallback DNS server, add it there as well.

```
[Resolve]
DNS=10.42.0.1
#FallbackDNS=
```

Enable `systemd-networkd` and then reboot for the changes to take effect:

```
$ sudo systemctl enable systemd-networkd
$ sudo reboot
```

Now start `tcpdump` so you can search for DHCP packets from the client Raspberry Pi:

```
$ sudo apt install tcpdump dnsmasq
$ sudo systemctl enable dnsmasq
$ sudo tcpdump -i eth0 port bootpc
```

Connect the client Raspberry Pi to your network and power it on. Check that the LEDs illuminate on the client after around 10 seconds, then you should get a packet from

the client "DHCP/BOOTP, Request from ..."

```
IP 0.0.0.0.bootpc > 255.255.255.255.bootps: BOOTP/DHCP, Request from b8:27:e
b...
```

Now you need to modify the `dnsmasq` configuration to enable DHCP to reply to the device. Press **CTRL + C** to exit the `tcpdump` program, then type the following:

```
$ echo | sudo tee /etc/dnsmasq.conf
$ sudo nano /etc/dnsmasq.conf
```

Then replace the contents of `dnsmasq.conf` with:

```
# Note: comment out port if you want DNS services for systems on the network.
port=0
dhcp-range=10.42.0.255,proxy
log-dhcp
enable-tftp
tftp-root=/tftpboot
pxe-service=0,"Raspberry Pi Boot"
```

Where the first address of the `dhcp-range` line is, use the broadcast address you noted down earlier.

Now create a `/tftpboot` directory:

```
$ sudo mkdir /tftpboot
$ sudo chmod 777 /tftpboot
$ sudo systemctl enable dnsmasq.service
$ sudo systemctl restart dnsmasq.service
```

Now monitor the `dnsmasq` log:

```
$ journalctl -f
```

You should see something like this:

```
raspberrypi dnsmasq-tftp[1903]: file /tftpboot/bootcode.bin not found
```

Next, you will need to copy the contents of the `boot` folder into the `/tftpboot` directory.

First, press **CTRL + C** to exit the monitoring state. Then type the following:

```
$ cp -r /boot/firmware/* /tftpboot
```

Since the `tftp` location has changed, restart `dnsmasq`:

```
$ sudo systemctl restart dnsmasq
```

Set up NFS root

This should now allow your Raspberry Pi client to attempt to boot through until it tries to load a root file system (which it doesn't have).

At this point, export the `/nfs/client1` file system created earlier, and the TFTP boot folder.

```
$ sudo apt install nfs-kernel-server
$ echo "/nfs/client1 *(rw,sync,no_subtree_check,no_root_squash)" | sudo tee
-a /etc/exports
$ echo "/tftpboot *(rw,sync,no_subtree_check,no_root_squash)" | sudo tee -a
/etc/exports
```

Restart `RPC-Bind` and the `NFS` server in order to have them detect the new files.

```
$ sudo systemctl enable rpcbind
$ sudo systemctl restart rpcbind
$ sudo systemctl enable nfs-kernel-server
$ sudo systemctl restart nfs-kernel-server
```

Edit `/tftpboot/cmdline.txt` and from `root=` onwards, and replace it with:


```
root=/dev/nfs nfsroot=10.42.0.211:/nfs/client1,vers=3 rw ip=dhcp rootwait
```

You should substitute the IP address here with the IP address you have noted down. Also remove any part of the command line starting with `init=`.

Finally, edit `/nfs/client1/etc/fstab` and remove the `/dev/mmcblk0p1` and `p2` lines (only `proc` should be left). Then, add the boot partition back in:

```
$ echo "10.42.0.211:/tftpboot /boot/firmware/ nfs defaults,vers=3 0 0" | sudo tee -a /nfs/client1/etc/fstab
```

If it doesn't boot on the first attempt, keep trying. It can take a minute or so for the Raspberry Pi to boot, so be patient.

Network boot using IPv6

[Edit this on GitHub](#)

There are 4 stages to booting a Raspberry Pi computer over the network:

1. The bootloader negotiates to get an IP address and the details of a TFTP server using DHCP.
2. The bootloader loads the firmware via TFTP and hands over the boot process to the firmware, passing it the details of the network.
3. The firmware loads the kernel and command line via TFTP.
4. The kernel boots the rest of the system, loading the root filesystem (`rootfs`) via NFS or some other mechanism.

The bootloader and firmware (stages 1 to 3) have been enhanced to support booting over IPv6.

IMPORTANT

IPv6 netboot is an **experimental alpha feature** and depending on feedback, we may need to change how it works in future. This only works on Raspberry Pi 4 and Compute Module 4.

How it works

To boot via IPv6 you need an updated version of the firmware (for example, `start4.elf`) and the bootloader. Using the latest release of Raspberry Pi OS and the latest stable bootloader should be sufficient.

NOTE

The commonly used `dnsmasq` DHCP server doesn't currently support the network boot parameters required for IPv6 network boot, so for the time being you will have to use a different DHCP server such as [ISC DHCP](#).

To mount `rootfs` over the network the [IPv4 netboot tutorial](#) suggests using `nfsroot`. This doesn't support IPv6, so another method is needed to mount `rootfs` over the network.

If your ISP and router don't support IPv6 you will be limited in what you can do.

Network addresses

The first thing the bootloader does is send a router solicitation to get the details of the network. The router responds with an advertisement packet identifying its ethernet address, which the bootloader might need if the TFTP server is on a different network.

The router advertisement includes a flag which tells it whether to use stateful (managed) or stateless (unmanaged) configuration for its IP address. Stateless configuration means that the device configures its own IP address. Currently the bootloader generates an address derived from its ethernet MAC address and a network prefix supplied by the router.

If the router indicates that stateful configuration is enabled DHCP is used to obtain the IP address of the device. This involves the device sending a solicitation request to a DHCP server which responds with an advertisement. The client then requests the address before getting a reply acknowledgement from the server.

DHCP Servers and clients identify themselves with variable length DUID (Device Unique ID). On the Raspberry Pi this is derived from the MAC address (DUID_LL).

TFTP address

Whether using stateless or stateful configuration, the DHCP server is used to obtain the TFTP server address. This is encoded in the `BOOTFILE-URL` parameter. We send the client architecture type value `0x29` to identify a device.

See [RFC 5970](#) and the [IANA Dynamic Host Configuration Protocol for IPv6](#) documentation.

Boot process

The device should now have an IP address and TFTP details. It downloads the firmware binary `start4.elf` from the TFTP server and continues running with this. The firmware is passed the IP address and TFTP server details so it can download the kernel and boot the rest of the system.

Kernel Boot

With IPv4 netboot, `nfsroot` is used to mount `rootfs` over the network. This doesn't support IPv6 so another solution is required. It might involve a small RAM file system that can mount the appropriate network location before switching to the proper `rootfs` contents.

NOTE

A mechanism to boot the Linux kernel with NFS via IPv6 is still to be demonstrated.

Test setup

If you want to try this out you will need another Raspberry Pi to act as the TFTP and DHCP server.

The TFTP server can in theory be on any routable network but the DHCP server has to be on the same network as the devices it will serve.

TFTP server

If you have a working IPv4 network boot setup, you can reuse the TFTP server in `dnsmasq` to supply the files (it can talk to both IPv4 and IPv6).

Alternatively you can use a standalone TFTP server like `tftpd-hpa`.

```
$ sudo apt install tftpd-hpa
$ sudo systemctl start tftpd-hpa
```

DHCP server

DHCP in IPv6 has changed a lot. We need DHCP to at least tell us the address of the TFTP server, which in this case is the same machine.

```
$ sudo apt install isc-dhcp-server
```

Modify the configuration in `/etc/default/isc-dhcp-server`

```
DHCPDv6_CONF=/etc/dhcp/dhcpd6.conf
INTERFACESv6="eth0"
```

In `/etc/dhcp/dhcpd6.conf` you need to specify the TFTP server address and setup a subnet. Here the DHCP server is configured to supply some made up unique local addresses (ULA). The host `test-rpi4` line tells DHCP to give a test device a fixed address.

```

not authoritative;

# Check if the client looks like a Raspberry Pi
if option dhcp6.client-arch-type = 00:29 {
    option dhcp6.bootfile-url "tftp://[fd49:869:6f93::1]/";
}

subnet6 fd49:869:6f93::/64 {
    host test-rpi4 {
        host-identifier option dhcp6.client-id 00:03:00:01:e4:5f:01:
20:24:0b;
        fixed-address6 fd49:869:6f93::1000;
    }
}

```

Your server has to be assigned the IPv6 address in `/etc/dhcpd.conf`

```

interface eth0
static ip6_address=fd49:869:6f93::1/64

```

Now start the DHCP server.

```
$ sudo systemctl restart isc-dhcp-server.service
```

Bootloader

Modify the configuration to tell it to attempt network boot via IPv6 rather than IPv4.

```

BOOT_ORDER=0xf21 # 2=Network boot
USE_IPV6=1 # Enable IPv6 network boot
BOOT_UART=1 # Debug

```

To revert to IPv4 network boot just remove the `USE_IPV6` line from `boot.conf`.

Router

To use IPv6 you really need a router and ISP that supports IPv6. There are sites on the internet that can check this for you or alternatively run the following command.

```

sudo apt install ndisc6
rdisc6 -1 eth0

```

This sends a router solicitation to your router asking for your network details such as the network prefix, router ethernet address and whether to use DHCP for addressing. If there's no response to this command it's likely your network and ISP only supports IPv4. If IPv6 is supported it's most likely that it will be configured to use stateless configuration where clients generate their own addresses.

```

Soliciting ff02::2 (ff02::2) on eth0...
Hop limit          :          64 (          0x40)
Stateful address conf. :          No
Stateful other conf.  :          Yes
Mobile home agent    :          No
Router preference     :          medium
Neighbor discovery proxy :          No
Router lifetime       :          180 (0x000000b4) seconds
Reachable time        : unspecified (0x00000000)
Retransmit time       : unspecified (0x00000000)

```

You might be able to configure your router for stateful configuration, which means it will use DHCP to obtain an IP address.

```

Hop limit          :          64 (          0x40)
Stateful address conf. :          Yes
Stateful other conf.  :          Yes
Mobile home agent    :          No
Router preference     :          medium
Neighbor discovery proxy :          No
Router lifetime       :          180 (0x000000b4) seconds
Reachable time        : unspecified (0x00000000)
Retransmit time       : unspecified (0x00000000)

```

Debugging

Logs and traces

If the boot UART is enabled, you should see something like this from the serial port. The lines starting RX6 indicate that IPv6 is in use.

Here dc:a6:32:6f:73:f4 is the MAC address of the TFTP server and it has an IPv6 address of fd49:869:6f93::1. The device itself has a MAC address e4:5f:01:20:24:0b and an IPv6 address of fd49:869:6f93::1000

```

Boot mode: NETWORK (02) order f
GENET: RESET_PHY
PHY ID 600d 84a2
NET_BOOT: e4:5f:01:20:24:0b wait for link TFTP6: (null)
LINK STATUS: speed: 100 full duplex
Link ready
GENET START: 64 16 32
GENET: UMAC_START 0xe45f0120 0x240b0000
RX6: 12 IP: 1 MAC: 1 ICMP: 1/1 UDP: 0/0 ICMP_CSUM_ERR: 0 UDP_CSUM_ERR: 0
NET fd49:869:6f93::1000 tftp fd49:869:6f93::1
RX6: 17 IP: 4 MAC: 4 ICMP: 2/2 UDP: 2/2 ICMP_CSUM_ERR: 0 UDP_CSUM_ERR: 0
TFTP_GET: dc:a6:32:6f:73:f4 fd49:869:6f93::1 ab5a4158/start4.elf

RX6: 17 IP: 4 MAC: 4 ICMP: 2/2 UDP: 2/2 ICMP_CSUM_ERR: 0 UDP_CSUM_ERR: 0
RX6: 18 IP: 5 MAC: 5 ICMP: 2/2 UDP: 3/3 ICMP_CSUM_ERR: 0 UDP_CSUM_ERR: 0
TFTP_GET: dc:a6:32:6f:73:f4 fd49:869:6f93::1 ab5a4158/config.txt

```

Finally the bootloader hands over to firmware which should load the kernel.

Stateful configuration

You can examine network activity with tcpdump.

```
$ sudo tcpdump -i eth0 -e ip6 -XX -l -v -vv
```

Below is an extract of a TCP dump where the router is configured to use stateful (DHCP) network configuration.

Device sends a router solicitation.

```

12:23:35.387046 e4:5f:01:20:24:0b (oui Unknown) > 33:33:00:00:00:02 (oui Unknown), ethertype IPv6 (0x86dd), length 70: (hlim 255, next-header ICMPv6 (58) payload length: 16) fe80::e65f:1ff:fe20:240b > ip6-allrouters: [icmp6 sum ok] ICMP6, router solicitation, length 16
    source link-address option (1), length 8 (1): e4:5f:01:20:24:0b
    0x0000: e45f 0120 240b

```

Router sends a response telling the device to use stateful configuration.

```

12:23:35.498902 60:8d:26:a7:c1:88 (oui Unknown) > 33:33:00:00:00:01 (oui Unknown), ethertype IPv6 (0x86dd), length 110: (hlim 255, next-header ICMPv6 (58) payload length: 56) bthub.home > ip6-allnodes: [icmp6 sum ok] ICMP6, router advertisement, length 56
    hop limit 64, Flags [managed, other stateful], pref medium, router lifetime 180s, reachable time 0ms, retrans timer 0ms
    rdnss option (25), length 24 (3): lifetime 60s, addr: bthub.home
    0x0000: 0000 0000 003c fe80 0000 0000 0000 628d
    0x0010: 26ff fea7 c188
    mtu option (5), length 8 (1): 1492
    0x0000: 0000 0000 05d4
    source link-address option (1), length 8 (1): 60:8d:26:a7:c1:88
    0x0000: 608d 26a7 c188

```

Device sends a DHCP solicitation.

```

12:23:35.502517 e4:5f:01:20:24:0b (oui Unknown) > 33:33:00:01:00:02 (oui Unknown), ethertype IPv6 (0x86dd), length 114: (hlim 255, next-header UDP (17) payload length: 60) fe80::e65f:1ff:fe20:240b.dhcpv6-client > ff02::1:2.dhcpv6-server: [udp sum ok] dhcpv6 solicit (xid=8cdd56 (client-ID hwaddr type 1 e45f0120240b) (IA_NA IAID:0 T1:0 T2:0) (option-request opt_59) (opt_61) (elapsed-time 0))

```

The DHCP server replies with an advertisement.

```

12:23:35.510478 dc:a6:32:6f:73:f4 (oui Unknown) > e4:5f:01:20:24:0b (oui Unknown), ethertype IPv6 (0x86dd), length 172: (flowlabel 0xad54d, hlim 64, next-header UDP (17) payload length: 118) fe80::537a:52c:c647:b184.dhcpv6-server > fe80::e65f:1ff:fe20:240b.dhcpv6-client: [bad udp csum 0xd886 -> 0x6d26!] dhcpv6 advertise (xid=8cdd56 (IA_NA IAID:0 T1:3600 T2:7200 (IA_ADDR fd49:869:6f93::1000 pltime:604800 vltime:259200)) (client-ID hwaddr type 1 e45f0

```

```
120240b) (server-ID hwaddr/time type 1 time 671211709 dca6326f73f4) (opt_59))
```

The device sends a request for an address and TFTP details to the DHCP server.

```
12:23:35.510763 e4:5f:01:20:24:0b (oui Unknown) > 33:33:00:01:00:02 (oui Unknown), ethertype IPv6 (0x86dd), length 132: (hlim 255, next-header UDP (17) payload length: 78) fe80::e65f:1ff:fe20:240b.dhcpv6-client > ff02::1:2.dhcpv6-server: [udp sum ok] dhcp6 request (xid=8cdd56 (client-ID hwaddr type 1 e45f0120240b) (server-ID hwaddr/time type 1 time 671211709 dca6326f73f4) (IA_NA IAID:0 T1:0 T2:0) (option-request opt_59) (opt_61) (elapsed-time 1))
```

The DHCP server replies, opt_59 is used to pass the address of the TFTP server.

```
12:23:35.512122 dc:a6:32:6f:73:f4 (oui Unknown) > e4:5f:01:20:24:0b (oui Unknown), ethertype IPv6 (0x86dd), length 172: (flowlabel 0xad54d, hlim 64, next-header UDP (17) payload length: 118) fe80::537a:52c:c647:b184.dhcpv6-server > fe80::e65f:1ff:fe20:240b.dhcpv6-client: [bad udp cksun 0xd886 -> 0x6826!] dhcp6 reply (xid=8cdd56 (IA_NA IAID:0 T1:3600 T2:7200 (IA_ADDR fd49:869:6f93::1000 pltime:604800 vltime:2592000)) (client-ID hwaddr type 1 e45f0120240b) (server-ID hwaddr/time type 1 time 671211709 dca6326f73f4) (opt_59))
```

The device sends a neighbour solicitation to the FTP server because it needs its MAC address.

```
12:23:36.510768 e4:5f:01:20:24:0b (oui Unknown) > 33:33:ff:00:00:01 (oui Unknown), ethertype IPv6 (0x86dd), length 86: (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::e65f:1ff:fe20:240b > ff02::1:ff00:1: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has fd49:869:6f93::1 source link-address option (1), length 8 (1): e4:5f:01:20:24:0b 0x0000: e45f 0120 240b
```

The FTP server replies with its MAC address.

```
12:23:36.510854 dc:a6:32:6f:73:f4 (oui Unknown) > e4:5f:01:20:24:0b (oui Unknown), ethertype IPv6 (0x86dd), length 86: (hlim 255, next-header ICMPv6 (58) payload length: 32) fd49:869:6f93::1 > fe80::e65f:1ff:fe20:240b: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is fd49:869:6f93::1, Flags [solicited, override] destination link-address option (2), length 8 (1): dc:a6:32:6f:73:f4 0x0000: dca6 326f 73f4
```

TFTP requests are made by the device which should now boot over the network.

```
12:23:36.530820 e4:5f:01:20:24:0b (oui Unknown) > dc:a6:32:6f:73:f4 (oui Unknown), ethertype IPv6 (0x86dd), length 111: (hlim 255, next-header UDP (17) payload length: 57) fd49:869:6f93::1000.61785 > fd49:869:6f93::1.tftp: [udp sum ok] 49 RRQ "ab5a4158/start4.elf" octet tsize 0 blksize 1024
```

Stateless configuration

Below is an extract of a tcp dump for a stateless (non-DHCP) network configuration.

The device sends a router solicitation.

```
12:55:27.541909 e4:5f:01:20:24:0b (oui Unknown) > 33:33:00:00:00:02 (oui Unknown), ethertype IPv6 (0x86dd), length 70: (hlim 255, next-header ICMPv6 (58) payload length: 16) fe80::e65f:1ff:fe20:240b > ip6-allrouters: [icmp6 sum ok] ICMP6, router solicitation, length 16 source link-address option (1), length 8 (1): e4:5f:01:20:24:0b 0x0000: e45f 0120 240b
```

The router replies with the network details.

```
12:55:27.834684 60:8d:26:a7:c1:88 (oui Unknown) > 33:33:00:00:00:01 (oui Unknown), ethertype IPv6 (0x86dd), length 174: (hlim 255, next-header ICMPv6 (58) payload length: 120) bthub.home > ip6-allnodes: [icmp6 sum ok] ICMP6, router advertisement, length 120 hop limit 64, Flags [other stateful], pref medium, router lifetime 180s, reachable time 0ms, retrans timer 0ms prefix info option (3), length 32 (4): 2a00:23c5:ee00:5001::/64, Flags [onlink, auto, router], valid time 300s, pref. time 120s 0x0000: 40e0 0000 012c 0000 0078 0000 0000 2a00 0x0010: 23c5 ee00 5001 0000 0000 0000 0000 prefix info option (3), length 32 (4): fd4d:869:6f93::/64, Flags
```

```
[onlink, auto, router], valid time 10080s, pref. time 2880s
0x0000: 40e0 0000 2760 0000 0b40 0000 0000 fd4d
0x0010: 0869 6f93 0000 0000 0000 0000 0000
rdnss option (25), length 24 (3): lifetime 60s, addr: bthub.home
0x0000: 0000 0000 003c fe80 0000 0000 0000 628d
0x0010: 26ff fea7 c188
mtu option (5), length 8 (1): 1492
0x0000: 0000 0000 05d4
source link-address option (1), length 8 (1): 60:8d:26:a7:c1:88
0x0000: 608d 26a7 c188
```

The device sends an information request to the DHCP multicast address asking for the TFTP details.

```
12:55:27.838300 e4:5f:01:20:24:0b (oui Unknown) > 33:33:00:01:00:02 (oui Unknown), ethertype IPv6 (0x86dd), length 98: (hlim 255, next-header UDP (17) payload length: 44) fe80::e65f:1ff:fe20:240b.dhcpv6-client > ff02::1:2.dhcpv6-server: [udp sum ok] dhcp6 inf-req (xid=e5e0a4 (client-ID hwaddr type 1 e45f0120240b) (option-request opt_59) (opt_61) (elapsed-time 0))
```

The DHCP server replies with the TFTP server details (opt_59).

```
12:55:27.838898 dc:a6:32:6f:73:f4 (oui Unknown) > e4:5f:01:20:24:0b (oui Unknown), ethertype IPv6 (0x86dd), length 150: (flowlabel 0xd1248, hlim 64, next-header UDP (17) payload length: 96) fe80::537a:52c:c647:b184.dhcpv6-server > fe80::e65f:1ff:fe20:240b.dhcpv6-client: [bad udp cksum 0xd870 -> 0x78bb!] dhcp6 reply (xid=e5e0a4 (client-ID hwaddr type 1 e45f0120240b) (server-ID hwaddr/time type 1 time 671211709 dca6326f73f4) (opt_59))
```

The device asks for the TFTP server MAC address since it can tell it's on the same network.

```
12:55:28.834796 e4:5f:01:20:24:0b (oui Unknown) > 33:33:ff:1d:fe:2a (oui Unknown), ethertype IPv6 (0x86dd), length 86: (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::e65f:1ff:fe20:240b > ff02::1:ff1d:fe2a: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2a00:23c5:ee00:5001:57f1:7523:2f1d:fe2a
source link-address option (1), length 8 (1): e4:5f:01:20:24:0b
0x0000: e45f 0120 240b
```

The FTP server replies with its MAC address.

```
12:55:28.834875 dc:a6:32:6f:73:f4 (oui Unknown) > e4:5f:01:20:24:0b (oui Unknown), ethertype IPv6 (0x86dd), length 86: (hlim 255, next-header ICMPv6 (58) payload length: 32) 2a00:23c5:ee00:5001:57f1:7523:2f1d:fe2a > fe80::e65f:1ff:fe20:240b: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is 2a00:23c5:ee00:5001:57f1:7523:2f1d:fe2a, Flags [solicited, override]
destination link-address option (2), length 8 (1): dc:a6:32:6f:73:f4
f4
0x0000: dca6 326f 73f4
```

The device starts making TFTP requests.

```
12:55:28.861097 e4:5f:01:20:24:0b (oui Unknown) > dc:a6:32:6f:73:f4 (oui Unknown), ethertype IPv6 (0x86dd), length 111: (hlim 255, next-header UDP (17) payload length: 57) 2a00:23c5:ee00:5001:57f1:7523:2f1d:fe2a.46930 > 2a00:23c5:ee00:5001:57f1:7523:2f1d:fe2a.tftp: [udp sum ok] 49 RRQ "ab5a4158/start4.elf" octet tsize 0 blksize 1024
```

You can view and edit the Raspberry Pi documentation source on [Github](#). Please read our [usage and contributions policy](#) before you make a Pull Request.

Raspberry Pi documentation is copyright © 2012-2026 Raspberry Pi Ltd and is licensed under a [Creative Commons Attribution-ShareAlike 4.0 International](#) (CC BY-SA) licence.

Some content originates from the [eLinux wiki](#), and is licensed under a [Creative Commons Attribution-ShareAlike 3.0 Unported](#) licence.

The terms HDMI, HDMI High-Definition Multimedia Interface, HDMI trade dress and the HDMI Logos are trademarks or registered trademarks of HDMI Licensing Administrator, Inc